

# Riscos & Prioridades de Melhoria

AutoJuri / JurisFlow · Branch **main** · Commit **744b6ef** · 2026-05-13

## Riscos do código atual

### 1 Integrações externas pouco testadas

`n8n_service.py` (23% de cobertura) e `suporte_email_service.py` (20%) são *single points of failure* do produto e estão praticamente sem rede de proteção. Cenários de erro (timeout, 5xx, schema inválido, falha SMTP) não são exercitados por nenhum teste.

### 2 `database.py` com 46% de cobertura e 978 linhas

Módulo monolítico que concentra conexão, sessões, usuários, contestações, dashboard e exemplares. Qualquer refatoração nessa camada tem risco alto de regressão silenciosa em produção.

### 3 `security.py` com 52% de cobertura

Funções de autenticação (`get_current_user`, helpers de hash legado) com baixa cobertura. Vetor de regressão silenciosa em segurança — mesmo havendo testes de segurança no nível de rota, caminhos internos do módulo permanecem não cobertos.

### 4 Frontend abaixo do threshold de 90%

Cobertura atual em **84,10%** — 5,9 p.p. abaixo do gate configurado no vitest. Componentes-chave (`App.jsx`, `MainPanelSection`, `RevisaoHumanaModal`) não possuem specs, e o pipeline `npm run test:coverage` falha hoje no gate de qualidade.

### 5 Função `contestar_por_peticao` com complexidade ciclomática 24 (rank D)

Concentra todo o fluxo crítico do MVP em uma única função: validação MIME, upload, extração de PDF, fallback OCR, persistência e disparo do orquestrador n8n. Complexidade alta + cobertura de rota em 79% deixa branches importantes não exercitados.

## Prioridades de melhoria

| # | Prioridade | Ação                                                                                                                                                                                                         | Impacto                                                                                |
|---|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 1 | ALTA       | Subir cobertura de <code>services/n8n_service.py</code> para $\geq 70\%$ com testes que mocam <code>httpx/requests</code> cobrindo timeout, 5xx e schema inválido.                                           | Destrava confiabilidade do orquestrador — é o módulo que dispara as chamadas Claude.   |
| 2 | ALTA       | Cobrir <code>services/suporte_email_service.py</code> (SMTP) nos caminhos felizes e nas três principais exceções ( <code>SupportEmailConfigError</code> , <code>SupportEmailServiceError</code> , fallback). | Garante que o canal de suporte ao usuário não falhe silenciosamente em produção.       |
| 3 | ALTA       | Refatorar <code>contestar_por_peticao</code> em $\geq 3$ funções menores (validação, extração, orquestração).                                                                                                | Reduz CC de 24 para $< 10$ por função e melhora drasticamente a testabilidade.         |
| 4 | MÉDIA      | Quebrar <code>database.py</code> em módulos por agregado: <code>db/usuario.py</code> , <code>db/sessao.py</code> , <code>db/contestacao.py</code> , <code>db/dashboard.py</code> .                           | Sobe MI, melhora cobertura natural e reduz risco de regressão em refatorações futuras. |
| 5 | MÉDIA      | Adicionar specs para <code>App.jsx</code> / <code>MainPanelSection.jsx</code> cobrindo o <i>golden path</i> (login → envio de petição → exibição no dashboard).                                              | Coloca o frontend acima dos 90% de threshold e libera o gate de cobertura no CI.       |
| 6 | MÉDIA      | Substituir <code>except Exception:</code> genéricos por exceções específicas + log estruturado nos 6 arquivos identificados.                                                                                 | Melhora drasticamente o diagnóstico de incidentes em produção.                         |

|   |       |                                                                                                                                                                                               |                                                                                   |
|---|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 7 | BAIXA | <p>           Extrair estado de <code>App.jsx</code> (47 <code>useState</code>) para hooks por domínio (<code>useAuth</code>, <code>useCases</code>, <code>useDashboard</code>).         </p> | <p>Reduz acoplamento e abre caminho para testes unitários do componente raiz.</p> |
|---|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|